## Today's agenda

- understanding sorting
- Problems on sorting
- Sorting techniques

AlgoPrep

**Sorting:** Arranging data in increasing/decreasing.
↳ on what Parameter.

Ex1: 2 4 10 15 27 → true

Ex2: 20 7 3 -5 -8 → true

Ex3: 1 2 3 7 4 9 6 → not sorted on the basis of value.

#factors: 1 2 2 2 3 3 4 → sorted on the basis of factor count.

- bubble sort
- selection sort
- insertion sort
- merge sort
- Quick sort
- bucket sort

→ develop

arr[4]: 3 1 9 8

↓

Arrays.sort(arr); → inc. order.

↳ T.C: O(NlogN)

↓

arr[4]: 1 3 8 9

AlgoPrep

## Q) Order of removal

↳ Given N elements at every step remove an array element. Cost to remove element : Sum of array elements present. Find min cost to remove all elements.

Note: Add cost first and then remove.

Ex1: arr[3] = {3 2 5}  (indices 0 1 2)

remove 3: 10
remove 2: 7
remove 5: 5
22

arr[3] = {3 2 5}  (indices 0 1 2)

remove 2: 10
remove 5: 8
remove 3: 3
21

Ex2: arr[4] = {4 6 2 7}  (indices 0 1 2 3)

remove 7: 4 + 6 + 2 + 7
remove 2: 4 + 6 + 2
remove 6: 4 + 6
remove 4: 4

minimize overall cost

max contribution = 0th index = min ele
2nd max contribution = 1st index = 2nd min ele.
3rd min ele.
⋮

→ Array should be sorted in inc. order.

AlgoPrep

arr[4] = { 4 6 2 7 }

0 1 2 3

↓

{ 2 4 6 7 }

0 1 2 3

4 times, 3 times, 2 times, 1 time

→ removal freq : N-i

//Pseudo code

```
int removalOrder (int arr[N]){
    Arrays.sort (arr);
    int ans = 0;

    for (int i = N-1; i >= 0; i--){
        ans = ans + (arr[i] * (N-i));
    }

    return ans;
}
```

T.C: O(NlogN + N)

≈ O(NlogN)

S.C: O(1)

AlgoPrep

```
int removalOrder (int arr[N]){
    Arrays.sort (arr);
    int ans = 0;

    for (int i = N-1; i >= 0; i--){
        ans = ans + (arr[i]*(N-i));
    }

    return ans;
}
```

arr[4]: 4 6 2 7

arr[4]: {2 4 6 7}

0 1 2 3

i: 0

ans = 0 + 7 + ~~246~~ 12 + ~~244~~ 12

8 ~~244~~ = 39

i = 3 : 2 + 4 + 6 + 7

i = 2 : 2 + 4 + 6

i = 1 : 2 + 4

i = 0 : 2

AlgoPrep

Q) Good Integer {only distinct}

↳ Given arr[n], Calculate no. of good integers.

An element is Said to be good if

{No. of element < ele :: ele itself}

i  arr[i]

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Ex1 : | -1 | -4 | 3 | 5 | -15 | 4 |
| # less : | 2 | 1 | 3 | 5 | 0 | 4 |

⇒ ans = 3

//idea1

↳ for every number, count smaller than that number and check if they are same. {nested loop}.

T.C: $O(n^2)$  S.C: $O(1)$

//idea2

↳ { -1 -4 3 9 -15 7 }

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| | -15 | -4 | -1 | 3 | 7 | 9 |
| | | | 2 | | 4 | 5 |

After sorting, Count of elements smaller than arr[i] = i

//Psuedo Code

```
int goodintegers (int arr[N]){
    Arrays.sort(arr);
    int count=0;

    for (int i=0; i<N; i++) {
        if (arr[i] == i){ count++; }
    }

    return count;
}
```

T.C: $O(N \log N)$

S.C: $O(1)$

AlgoPrep

{No. of element < ele :: ele itself}

**Good Integers** : { Data can repeat }

| | 0 | 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|---|---|
| Ex1: | 0 | 2 | 2 | 3 | 3 | 8 |
| #less : | 0 | 1 | 1 | 3 | 3 | 5 |

→ ans = 3

→ edge case

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 ← i |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ex2: | -4 | -4 | 2 | 2 | 5 | 5 | 5 | 5 | 8 | 8 | 8 | 10 | 17 |
| #less : | 0 | 0 | 2 | 2 | 4 | 4 | 4 | 4 | 8 | 8 | 8 | 11 | 12 |

After Sorting

↳ ans = 5

**OBS1:** All the occurrences of same number will be either good or bad.

**OBS2:** if ele is 1st occ. → arr[i] != arr[i-1]

⇓

No. of ele < ele == i

i == arr[i]

**OBS3:** if Ele is not the 1st occ.

//Psuedo code

```
int goodintegerduplicate (int arr[N]){
    Arrays.sort (arr);
    int count = 0;

    int lesscount = 0;
    for (int i=1; i<N; i++){

        if (arr[i] != arr[i-1]){ //first occ.
            lesscount = i;
        }
        else{ // next occ.

        }

        if (arr[i] == lesscount) { count ++; }

    }

}
```

T.C: O(N logN)

S.C: O(1)

AlgoPrep

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Ex2: | -4 | -4 | 2 | 2 | 5 | 5 | 5 | 5 | 8 | 8 | 8 | 10 | 17 |
| //less : | 0 | 0 | 2 | 2 | 4 | 4 | 4 | 4 | 8 | 8 | 8 | | |

```
int goodintegerduplicate (int arr[N]){
    Arrays.sort (arr);
    int count = 0;
    if (arr[0] == 0){count ++;}
    int lesscount = 0;
    for (int i = 1; i < N; i++){

        if (arr[i] != arr[i-1]){ //first occ.
            lesscount = i;
        }
        else{ // next occ.
            nothing
        }

        if (arr[i] == lesscount){count ++;}
    }
}
```

count = 0 1 2 3 4 5

lesscount = 0 1 4 8

## //Sorting techniques

### ① Bubble Sort

↳ Sort the array in asc. order but we can swap adjacent elements only.

| | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| arr[8]: | 5 | 7 | 5 | 4 | 10 | -2 | 6 | 3 |

**iter 0 :** $\{5\ \cancel{7}\ \cancel{5}\ \cancel{4}\ \cancel{10}\ \cancel{-2}\ \cancel{6}\ \cancel{3}\}$ → $\{0, N-2\}$

$5\ \cancel{7}_4\ 7\ -2\ \cancel{10}_6\ \cancel{10}_3\ 10$

$\{5\ 5\ 4\ 7\ -2\ 6\ 3\ 10\}$

**iter 1 :** $\{5\ \cancel{5}\ \cancel{4}\ \cancel{7}\ \cancel{-2}\ \cancel{6}\ \cancel{3}\ 10\}$ → $\{0, N-3\}$

$4\ 5\ -2\ \cancel{7}_6\ \cancel{7}_3\ 7$

$\{5\ 4\ 5\ -2\ 6\ 3\ 7\ 10\}$

⋮

**iter 2 :** $\{0, N-4\}$

⋮

**iter 6**

iter 7 ?? ↳ N length → N-1 iteration

$\{0, N-2\}$

## //Psuedo code

T.C: $O(N^2)$

S.C: $O(1)$

```
void bubblesort (int arr[N]){

                    i < N-1
                    ||
    for (i=0; i<=N-2 ; i++){

        for (int j=0; j<=N-2-i ; j++){
            if (arr[j] > arr[j+1]){
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

AlgoPrep

arr[8] = { 5 7 5 4 10 -2 6 3 }

```
void bubbleSort (int arr[N]){

    for (i=0; i<=N-2; i++){

        for (int j=0; j<=N-2-i; j++){
            if (arr[j] > arr[j+1]){
                int temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```

i<N-1

i=0

j→{0, 6}

{ 5 7 5 4 10 -2 6 3 }

5 7 4 7 -2 10 6 10 3

{ 5 5 4 7 -2 6 3 10 }

i=1

j→{0, 5}

{ 5 5 4 7 -2 6 3 10 }

4 5 -2 7 6 3 7

{ 5 4 5 -2 6 3 7 10 }

AlgoProp